OBJECT ORIENTED SOFTWARE ENGINEERING

L	T	P	C
3	0	2	4

Course Objectives:

- To understand object-oriented concepts and software engineering principles
- To learn UML for modeling software systems
- To apply OOAD methodology for real-world problems
- To design, implement, and evaluate object-oriented systems

UNIT I PROCESS MODELS AND REQUIREMENT ANALYSIS 9

Software Engineering – Definition –Need- Software Process Models: Waterfall, Incremental, Spiral, component based, Agile – SDLC – Procedural vs Object-Oriented Paradigm – Introduction to OOAD. Requirement analysis and specification – Requirements gathering and analysis – Software Requirement Specification

UNIT II UML AND STRUCTURAL MODELING 9

Introduction to UML – UML Architecture – UML Diagrams Overview – Structural Modeling: Class Diagram, Object Diagram, Package Diagram, Component Diagram – Relationships: Association, Aggregation, Composition, Generalization, Dependency.

UNIT III UML BEHAVIORAL MODELING 9

Use Case Diagrams – Actors, Use Cases, Relationships – Sequence Diagram – Communication Diagram – Activity Diagram – State Machine Diagram – Interaction Overview Diagram.

UNIT IV OBJECT ORIENTED ANALYSIS AND DESIGN 9

OOAD Process – Requirements Analysis – Use Case Driven Approach – Domain Modeling – Design Principles (SOLID) – Introduction to Design Patterns :Creational-Singleton, Structural: Adaptor, Behavioural: strategy – Mapping Analysis to Design.

UNIT V IMPLEMENTATION, TESTING AND CASE STUDIES 9

Mapping UML to Code – Implementation of Class Diagrams – Software Testing: Unit Testing, Integration Testing, System Testing – Refactoring – Reverse Engineering and Forward Engineering – Case Studies using UML Tools.

45 Periods

LIST OF EXPERIMENTS: **30 periods**

1. Identify a software system that needs to be developed.
2. Document the Software Requirements Specification (SRS) for the identified system.
3. Identify use cases and develop the Use Case model.
4. Identify the conceptual classes and develop a Domain Model and also derive a Class Diagram from that.
5. Using the identified scenarios, find the interaction between objects and represent them using UML Sequence and Collaboration Diagrams
6. Draw relevant State Chart and Activity Diagrams for the same system.
7. Implement the system as per the detailed design
8. Test the software system for all the scenarios identified as per the usecase diagram
9. Improve the reusability and maintainability of the software system by applying appropriate design patterns.
10. Implement the modified system and test it for various scenarios.

SUGGESTED DOMAINS FOR MINI-PROJECT:

1. Passport automation system.
2. Book bank
3. Exam registration
4. Stock maintenance system.
5. Online course reservation system
6. Airline/Railway reservation system
7. Software personnel management system
8. Credit card processing
9. e-book management system
10. Recruitment system
11. Foreign trading system
12. Conference management system
13. BPO management system
14. Library management system
15. Student information system **TOTAL:75 PERIODS**

COURSE OUTCOMES (COs):

Upon completion of the course, students will be able to:

- **CO1:** Explain software engineering and object-oriented concepts *(K2)*
- **CO2:** Apply UML diagrams for system modeling*(K3)*
- **CO3:**Analyze system requirements using OO techniques *(K4)*
- **CO4:** Design object-oriented systems using OOAD principles *(K5)*
- **CO5:** Develop and evaluate software systems using UML tools *(K6)*

TEXTBOOKS:

1. Bernd Bruegge ,Allen H. Dutoit, "Object-Oriented Software Engineering Using UML, Patterns, and Java", Pearson Education, 3rd edition, 2014.
2. Larman Craig, "Applying UML and Patterns ", Pearson Education 3rd Edition, 2015
3. Roger S. Pressman, "Object-Oriented Software Engineering: An Agile Unified Methodology", Mc Graw-Hill International, First Edition, 2014.

REFERENCES:

1. Greedy Booch, "The Unified Modeling Language User Guide" , Addison-Wesley Object Technology Series, 2005
2. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software", Pearson, 2015